# COMP 3311
# DATABASE MANAGEMENT SYSTEMS

## LECTURE 3
## DATABASE DESIGN:
## ENTITY-RELATIONSHIP (E-R) DATA MODEL

# E-R DATA MODEL: CONSTRAINTS

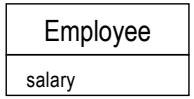
A constraint is a logical restriction or a property of data that for any set of data values:

- we can determine whether the constraint is true or false;
- we expect the constraint to be always true;
- a DBMS can enforce the constraint.

**Examples:**

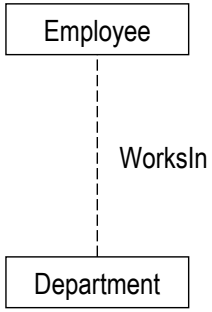
**on attributes**

salary is between $0 and $100,000

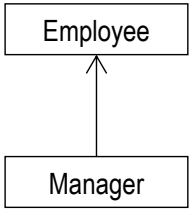



**on relationships**

every employee works in *at most* one department




not every employee is a manager




☞ **Constraints add additional semantics (meaning) to data**

**(so as to more accurately reflect the application requirements).**

# E-R DATA MODEL CONSTRAINTS: OBSERVATIONS

- Not all constraints on data can be represented by the E-R data model.

  ☞ **Some constraint will need to be enforced by the applications.**

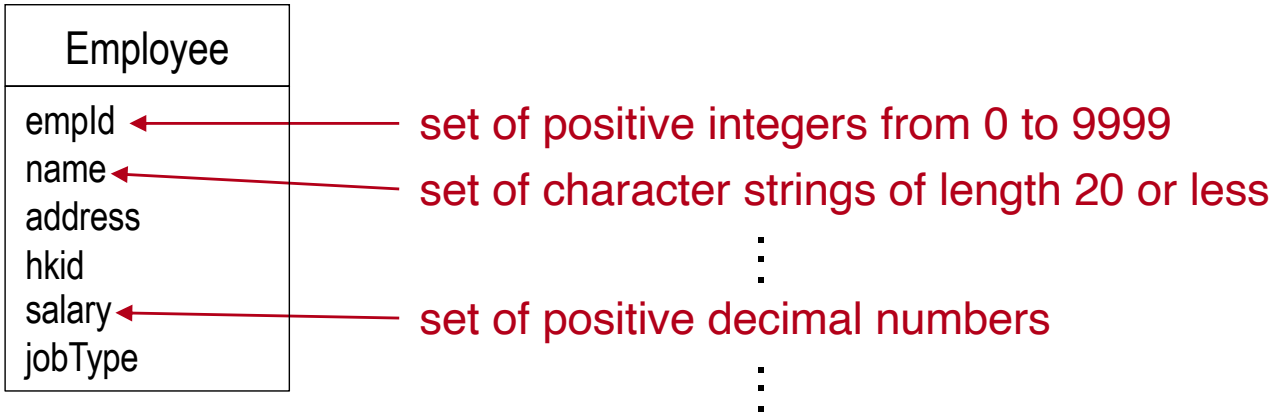
- Even though a constraint could be represented by the E-R data model, it may not be desirable to do so as the constraint representation may be quite complex and may reduce the modeling clarity.

  ☞ **Some constraints are best left to the applications to enforce.**

☞ **The database designer needs to decide which constraints the schema should enforce and which constraints the applications should enforce.**

# ATTRIBUTE CONSTRAINTS: DOMAIN

**A *domain constraint* restricts an attribute to have only certain values.**




**Example**

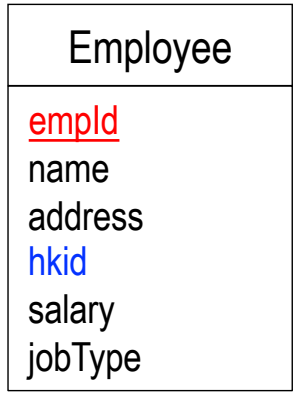
The integer constraint for empId can be specified as a data type.

The range of values constraint for empId must be specified as a predicate.

A domain constraint can be specified as a data type for an attribute and/or a predicate (logical condition) that restricts the values of an attribute.

# ATTRIBUTE CONSTRAINTS: KEY

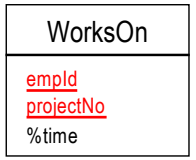
- If the values of some attributes uniquely identify an entity instance, then they are a candidate key for the entity.
  - ☞ A candidate key is a minimal set of attributes (i.e., all attributes are needed) that uniquely identifies an entity instance.
  - ☞ An entity may have more than one candidate key.




- Usually, one candidate key is selected by the database designer to be the primary key (or simply key) of an entity.
  - ☞ **This has enforcement implications for implementation.**

    **primary key** ⟹ uniqueness is automatically enforced by a DBMS
    **other candidate keys** ⟹ uniqueness is not automatically enforced by a DBMS
    (unless the database designer explicitly specifies uniqueness)

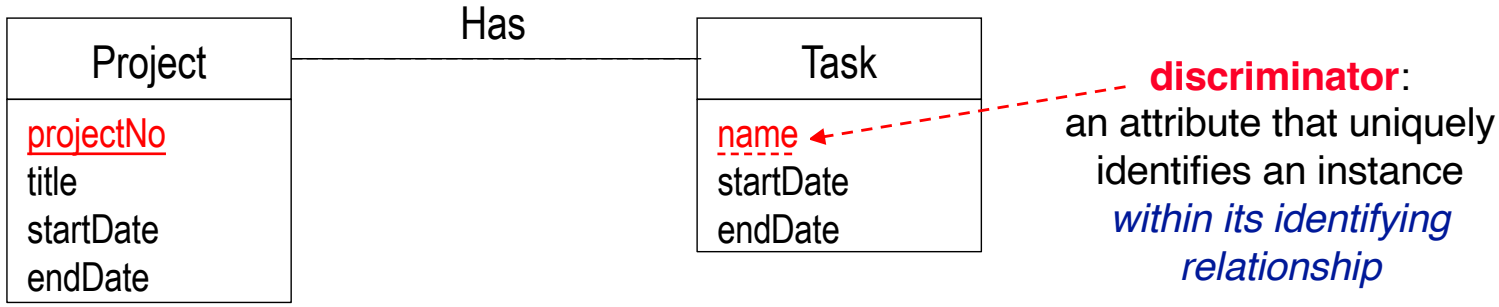
- A candidate/primary key can be composed of a set of attributes ⟹ composite candidate/primary key.



# STRONG ENTITY VS. WEAK ENTITY

**Strong entity**: An entity that has a primary key.

**Weak entity**: An entity that does not have a primary key.




- A weak entity must be associated with an identifying entity, to exist and be meaningful.
  - ☞ A weak entity depends on its identifying entity for its existence.
- The relationship associating a weak entity and an identifying entity is called the identifying relationship (shown as a solid line).
- A discriminator, *if present*, uniquely identifies a weak entity instance *in the context of* its identifying relationship.

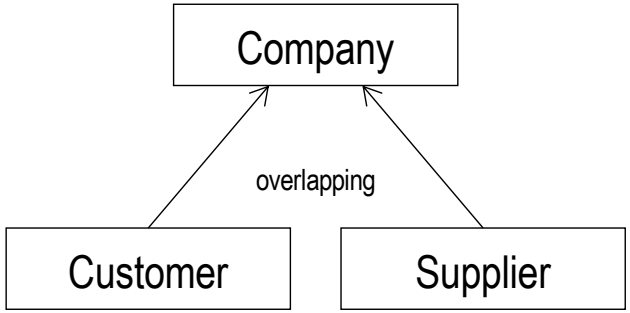
# ENTITY GENERALIZATION CONSTRAINTS: COVERAGE

## Disjointness

### (a) overlapping

A supertype instance can relate to more than one subtype.

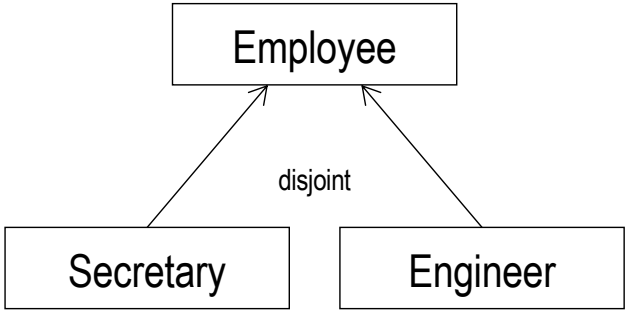
E.g., a given company can be both a customer and a supplier.




### b) disjoint

A supertype instance can relate to at most one subtype.

E.g., a given employee can be either a secretary or an engineer, but not both.



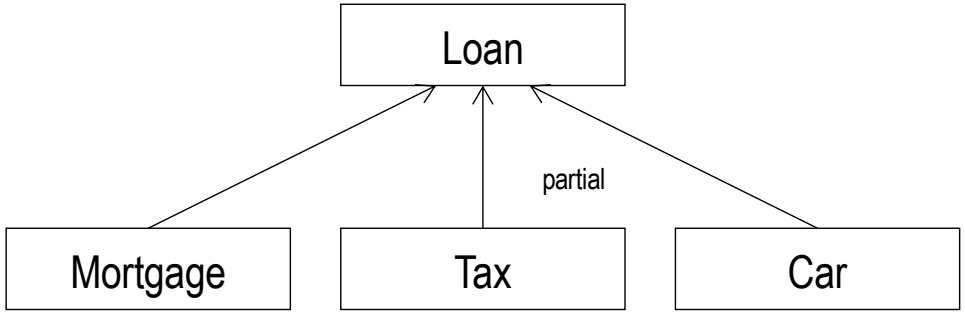
# ENTITY GENERALIZATION CONSTRAINTS: COVERAGE (CONT'D)

## Completeness

### a) partial

A supertype instance does not need to relate to any of the subtypes.

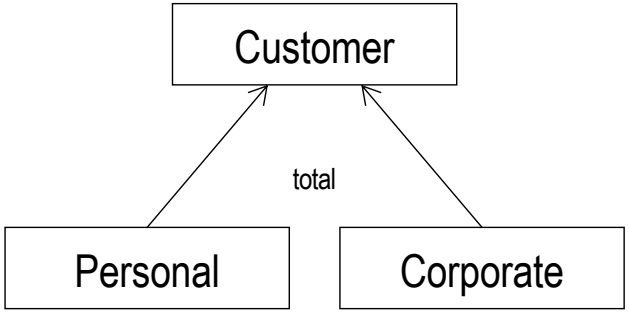
E.g., a loan does not need to be a mortgage (loan) or a tax (loan) or a car (loan)—there are other kinds of loans.




### (b) total

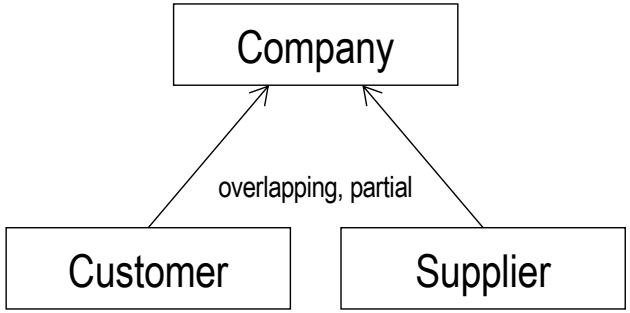
A supertype instance must relate to at least one of the subtypes.

E.g., a given customer must be either a personal or a business customer.

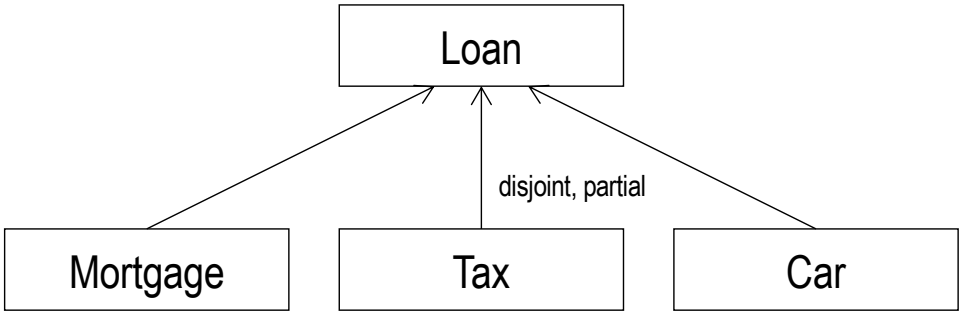


# ENTITY GENERALIZATION CONSTRAINTS: COVERAGE (CONT'D)

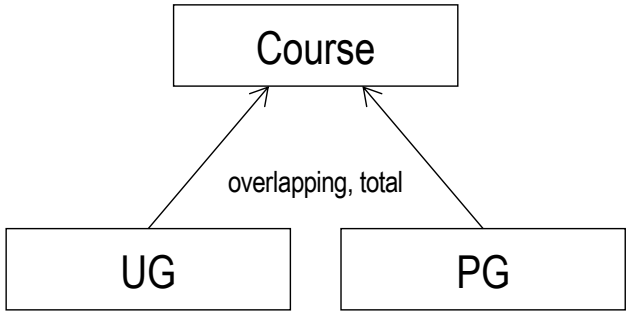
**overlapping, partial**



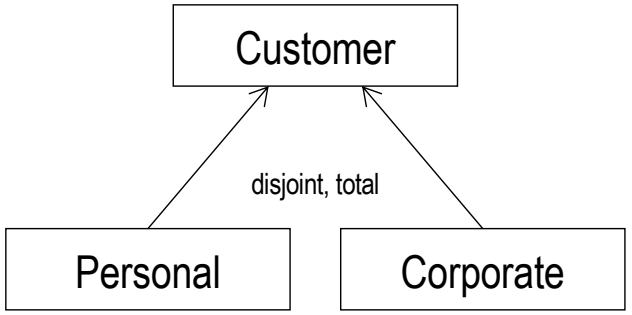

**disjoint, partial**




**overlapping, total**



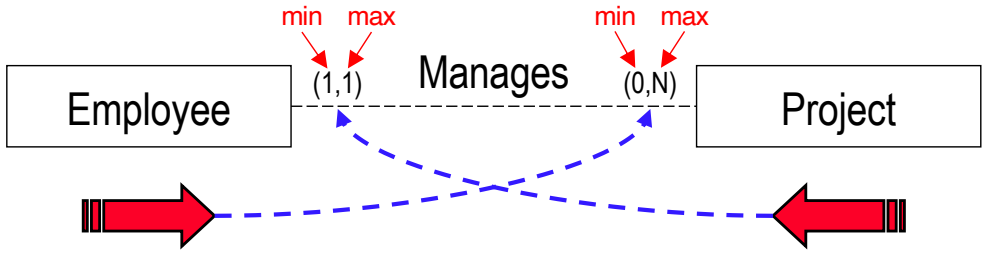

**disjoint, total**




☞ **Coverage is specified as one from disjointness (when there is more than one subtype) and one from completeness.**

# RELATIONSHIP CONSTRAINTS: CARDINALITY & PARTICIPATION

***Cardinality*** **specifies the maximum number and** ***participation*** **specifies the minimum number of relationship instances in which an entity instance may participate in a specified relationship type.**



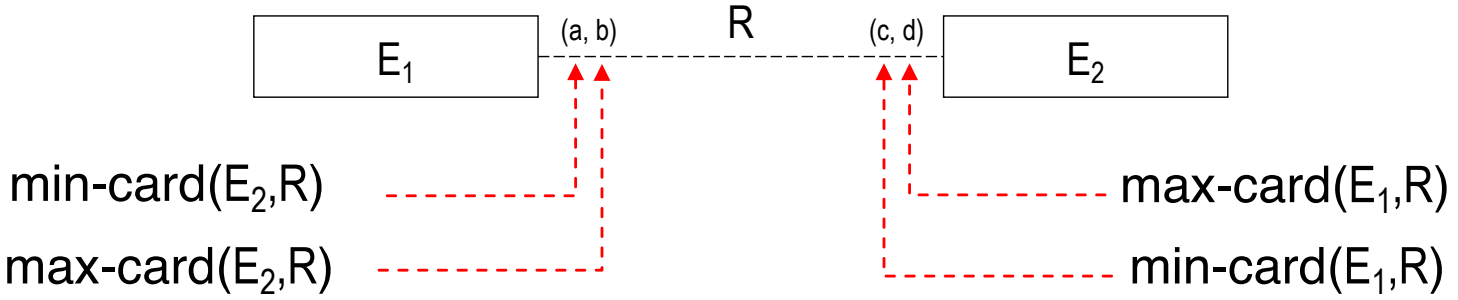

**For a given project, how many employees can manage it?**

- ☞ Each project is managed by one and only one employee.

**For a given employee, how many projects can he/she manage?**

- ☞ An employee does not have to manage any project, but may manage several (i.e., an unknown number of) projects.

# RELATIONSHIP CONSTRAINTS: CARDINALITY & PARTICIPATION




**minimum cardinality (min-card) ⟹ participation constraint**

min-card($E_1$,R): The *minimum* number of relationship instances in which each entity of $E_1$ *must* participate in R.

min-card($E_1$,R) = 0 ⟹ partial participation (*does not have to* participate)

min-card($E_1$,R) > 0 ⟹ total participation (*must* participate)

**maximum cardinality (max-card) ⟹ cardinality constraint**

max-card($E_1$,R): The *maximum* number of relationship instances in which each entity of $E_1$ *may* participate in R.

# RELATIONSHIP CONSTRAINTS: SPECIAL MAXIMUM CARDINALITIES

max-card($E_1$,R) = 1 and max-card($E_2$,R) = 1

- ☞ **one-to-one relationship (1:1)**

max-card($E_1$,R) = 1 and max-card($E_2$,R) = N

- ☞ **one-to-many relationship (1:N)**

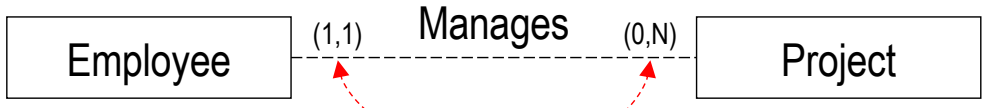
max-card($E_1$,R) = N and max-card($E_2$,R) = 1

- ☞ **many-to-one relationship (N:1)**

max-card($E_1$,R) = N and max-card($E_2$,R) = N

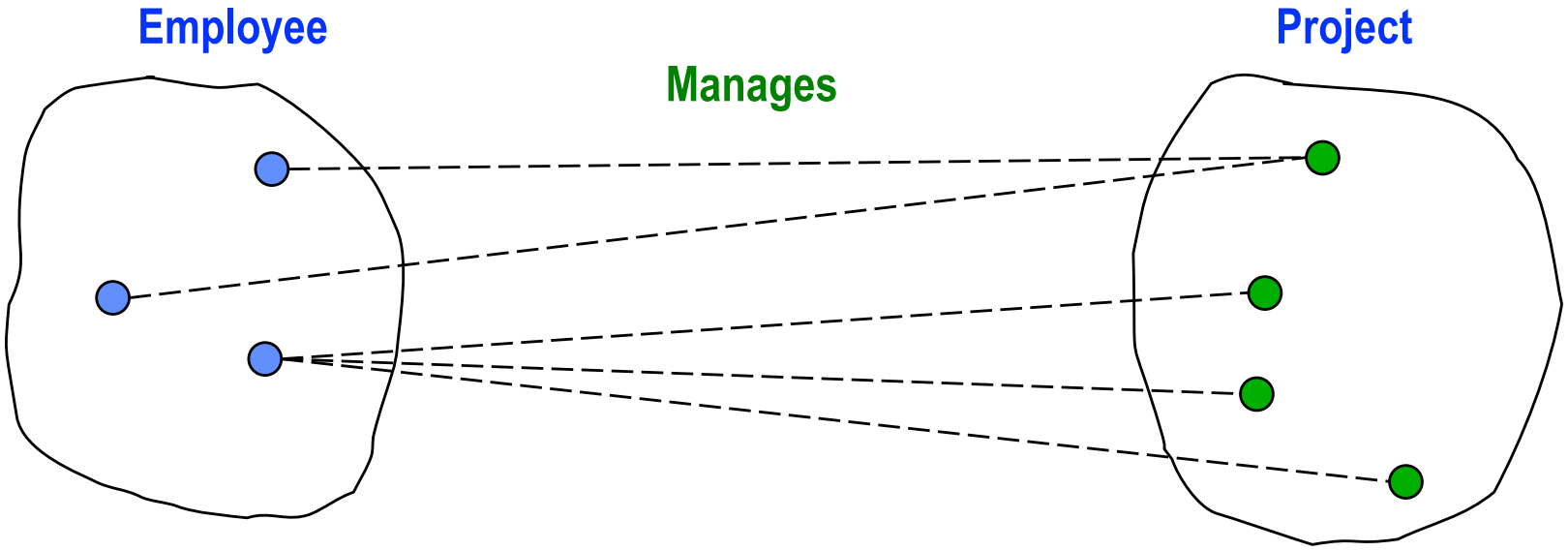
- ☞ **many to many relationship (N:M)**

# RELATIONSHIP CONSTRAINTS: CARDINALITY AND PARTICIPATION



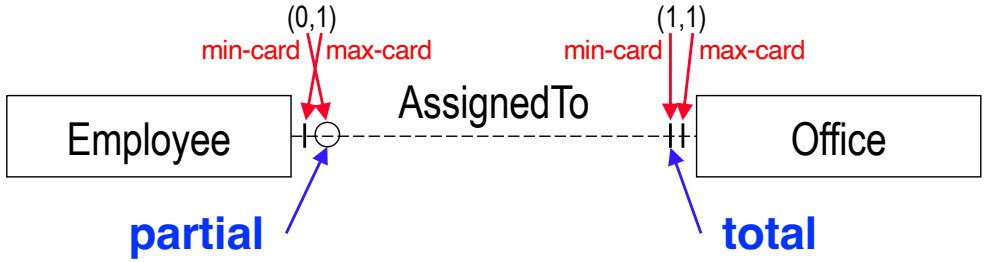

**An employee does not have to manage a project but may manage several projects.**

**Every project must be managed by an employee and at most one employee.**

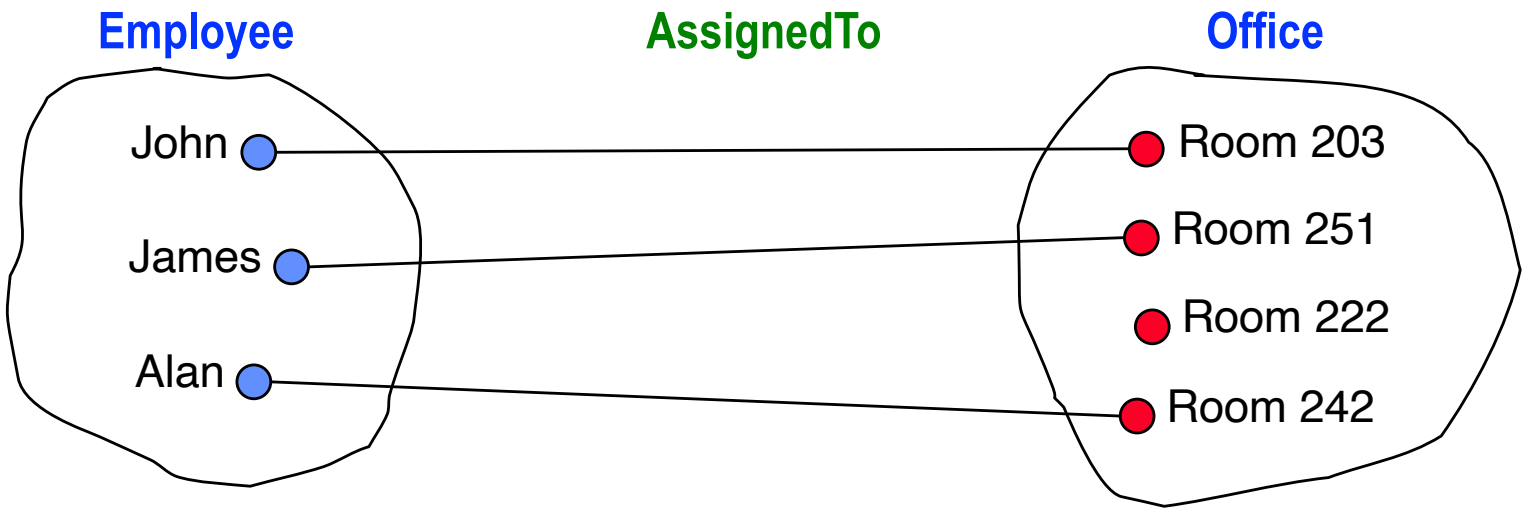


# RELATIONSHIP CONSTRAINTS: EXAMPLE CARDINALITY AND PARTICIPATION

## one-to-one (1:1) relationship

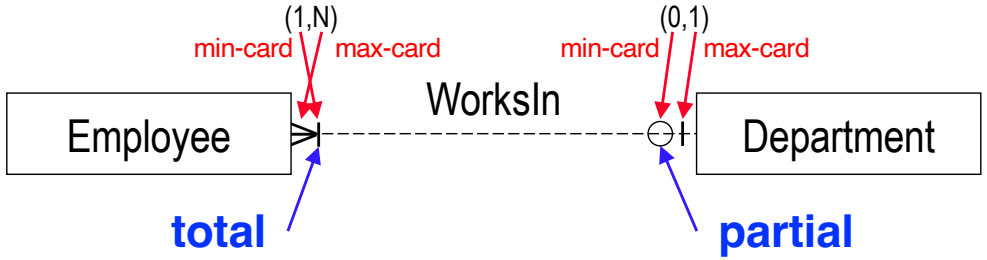



information engineering (crow foot) notation

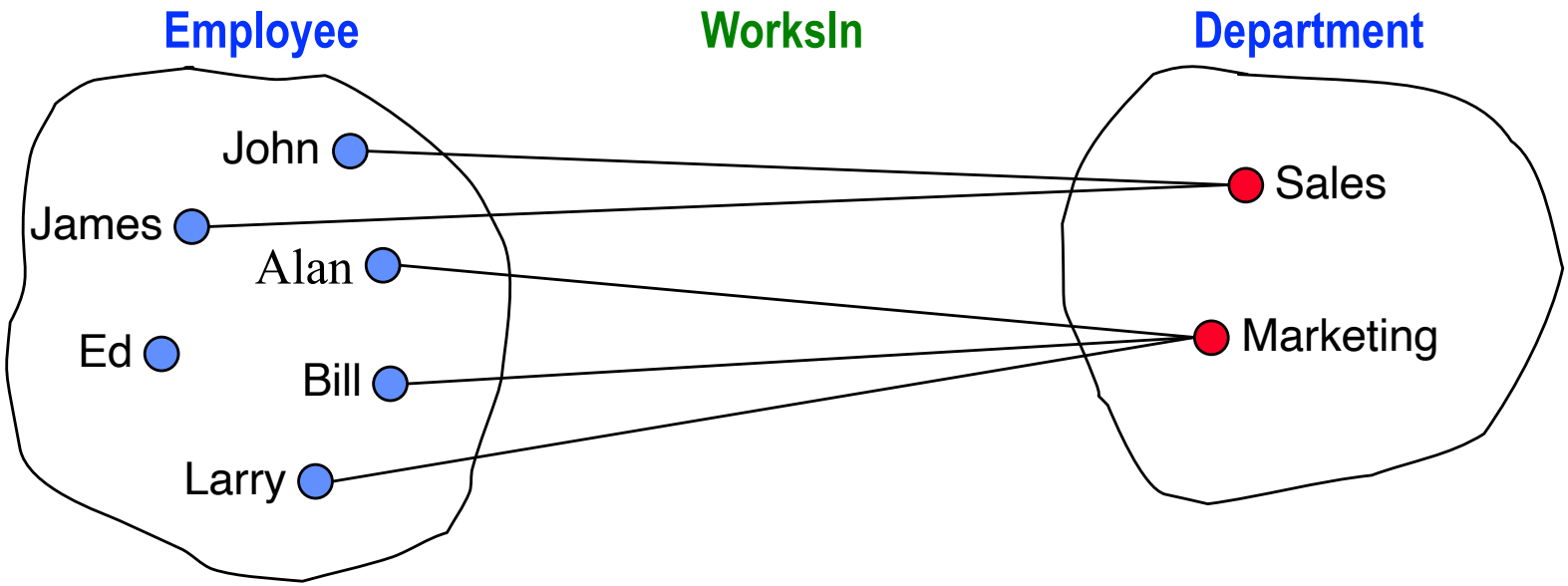


# RELATIONSHIP CONSTRAINTS: EXAMPLE CARDINALITY AND PARTICIPATION

## one-to-many (1:N) relationship

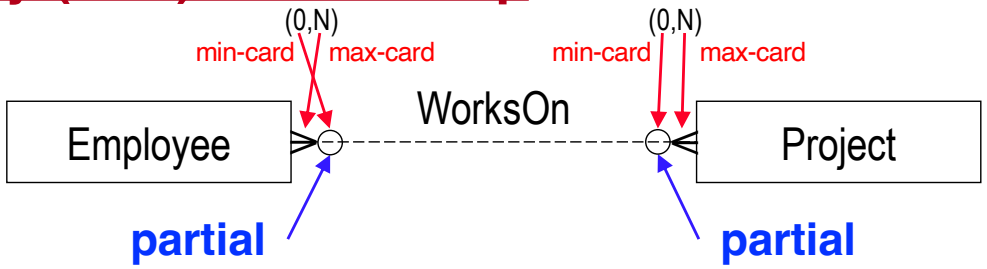



information engineering (crow foot) notation

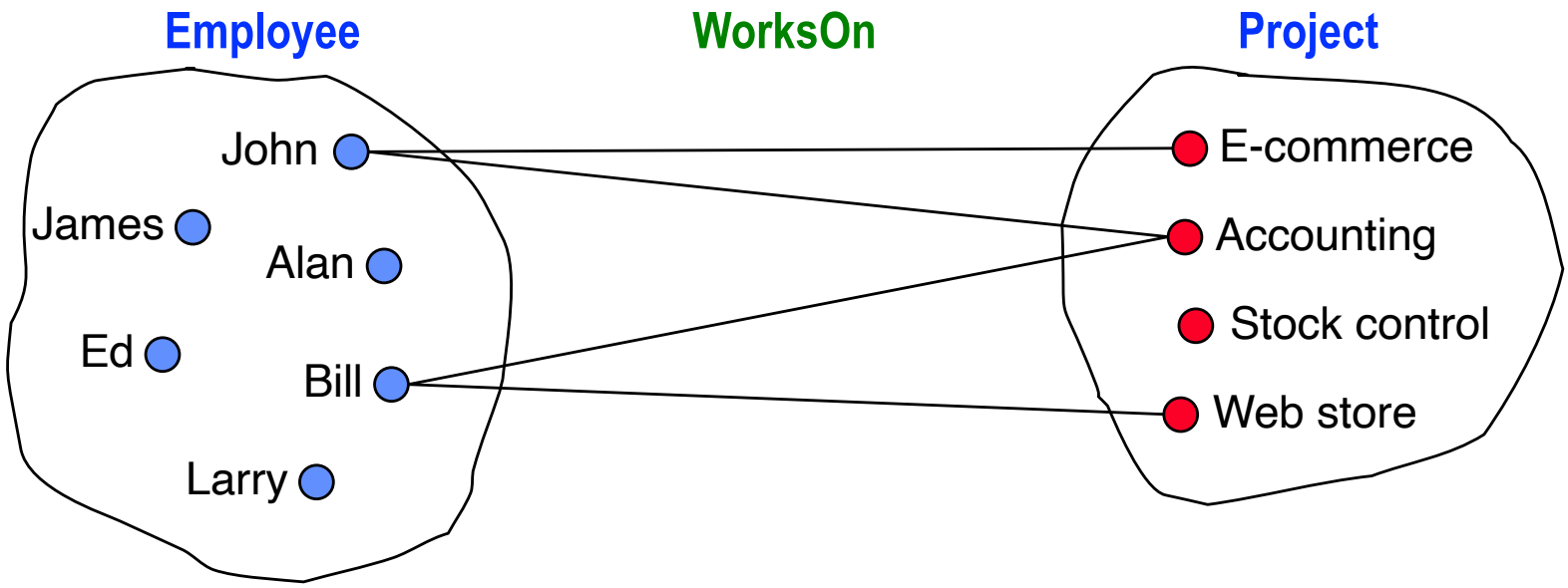


# RELATIONSHIP CONSTRAINTS: EXAMPLE CARDINALITY AND PARTICIPATION

## many-to-many (N:M) relationship



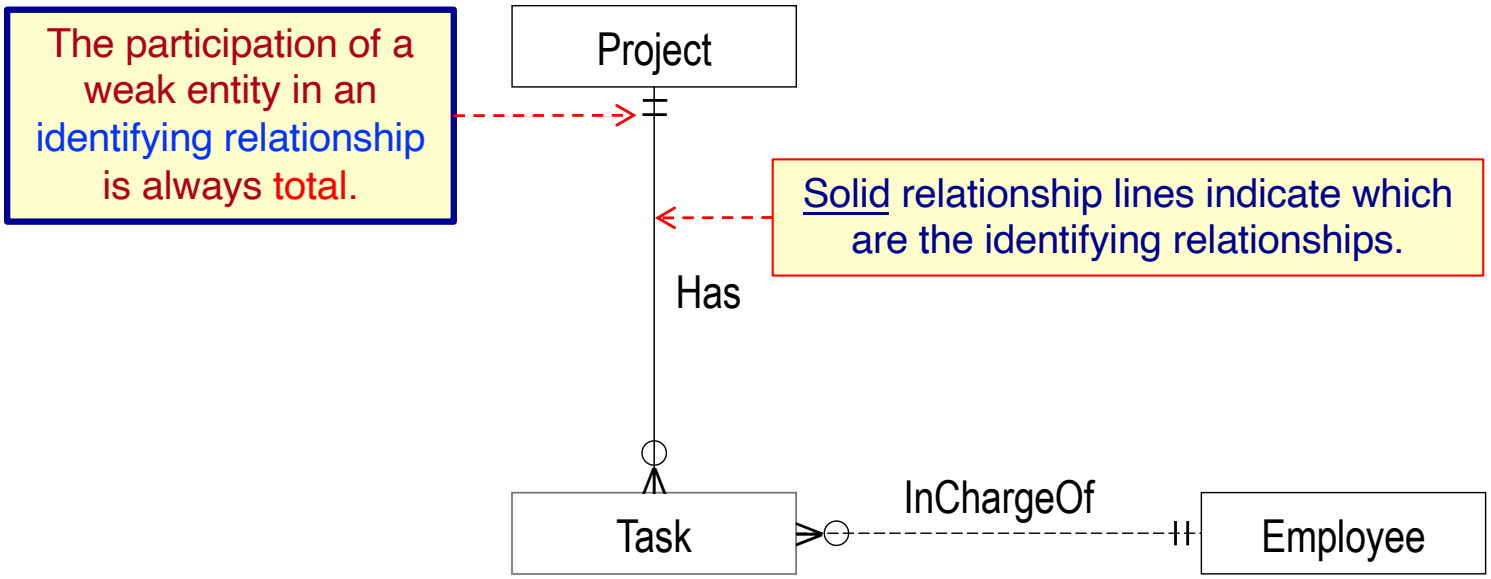

information engineering (crow foot) notation



# RELATIONSHIP CONSTRAINTS: WEAK ENTITY PARTICIPATION

- A weak entity may be related to more than one identifying entity but may depend on only some of these entities for its existence.

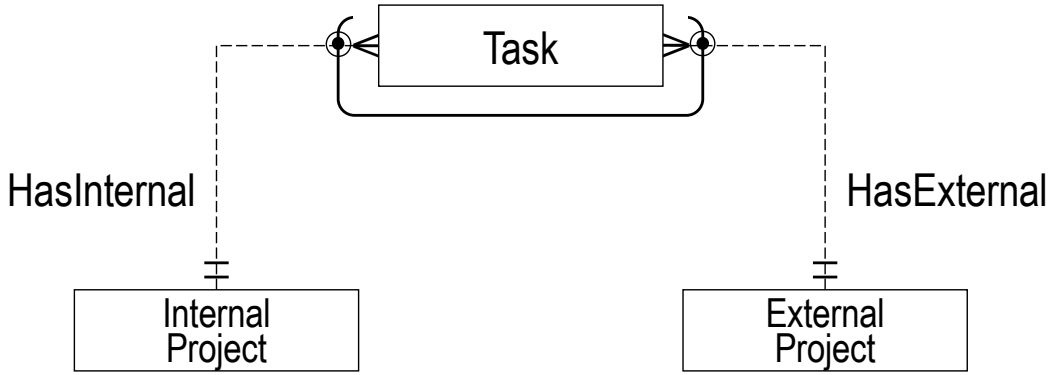
☞ **Not all the related entities need to be identifying entities.**



# RELATIONSHIP CONSTRAINTS: EXCLUSION

**An *exclusion (XOR) constraint* specifies that at most one entity instance, from among several related entity types, can participate in a relationship with a single "root" entity.**

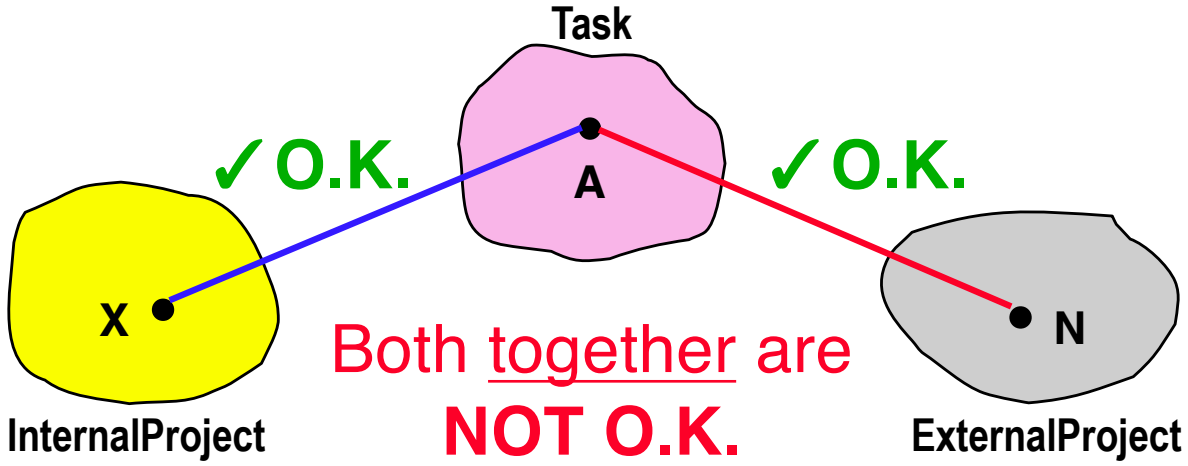
**Example:** A task can be related to *either* an internal project *or* an external project, *but not both*.



# RELATIONSHIP CONSTRAINTS: EXCLUSION (CONT'D)

**An *exclusion (XOR) constraint* specifies that at most one entity instance, from among several related entity types, can participate in a relationship with a single "root" entity.**

**Example:** A task can be related to *either* an internal project *or* an external project, *but not both*.

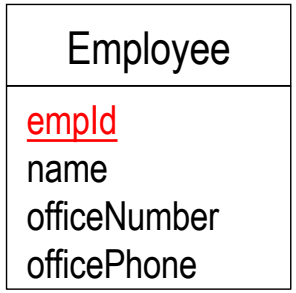


# DESIGN CHOICE: ENTITY VERSUS ATTRIBUTE

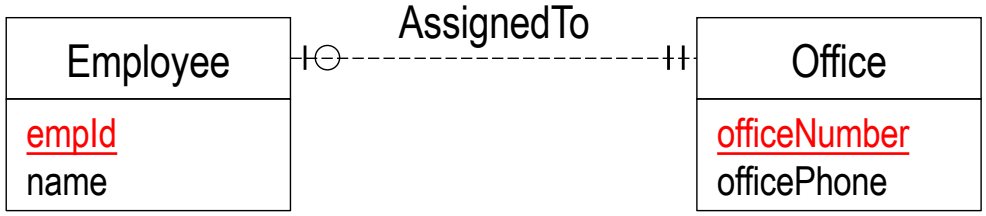
entity: When several properties can be associated with the concept.

attribute: When the concept has a simple atomic structure or no property of interest.

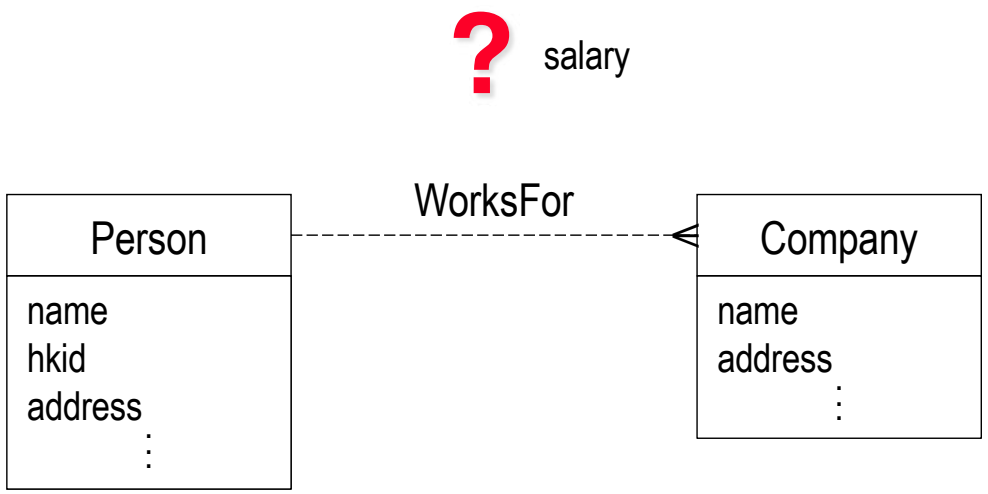
**Office as attribute**




**Office as entity**



# DESIGN CHOICE: PLACING AN ATTRIBUTE




**Where to place salary?**

**Relationship attributes may be needed when an attribute value exists only when a relationship exists.**

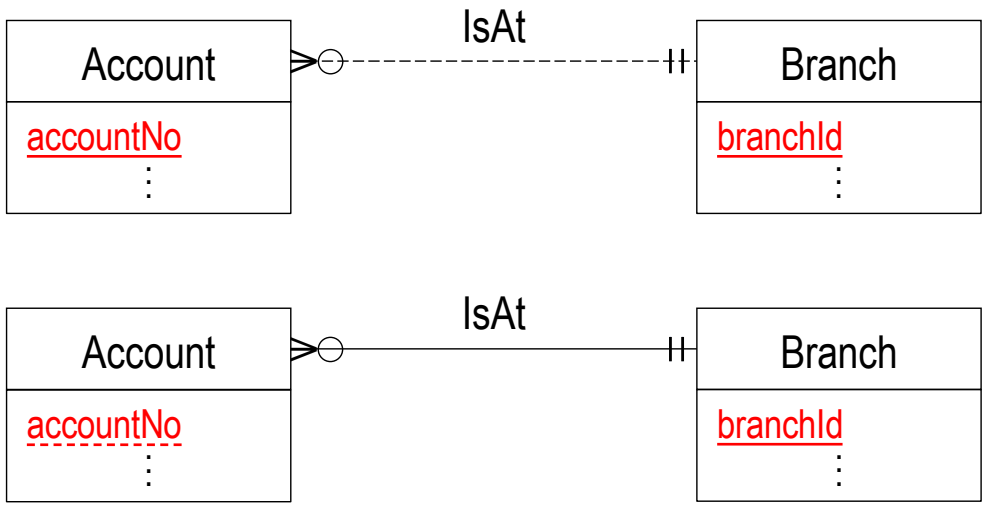
# DESIGN CHOICE: STRONG VERSUS WEAK ENTITY

strong entity: When the concept can be uniquely identified in the application domain (i.e., it has a key).

weak entity: When the concept has no unique identifier.

**Suppose an account must be associated with exactly one branch, within a branch account numbers are unique, and two different branches can have accounts with the same number.**

**Should Account be a strong or weak entity?**

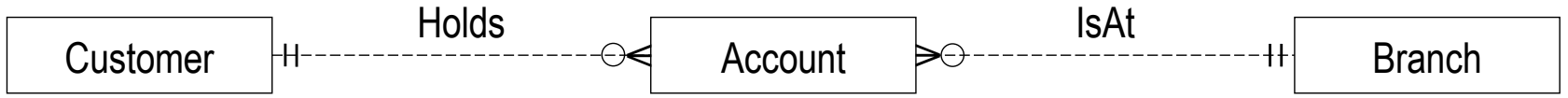


# DESIGN CHOICE: ENTITY VERSUS RELATIONSHIP

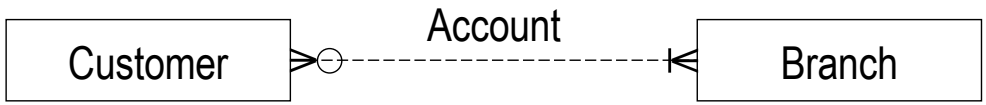
entity: When the concept represents something distinct in the application domain with several properties.

relationship: When the concept is not a distinct application domain concept and/or has no property of interest.

## Account as entity



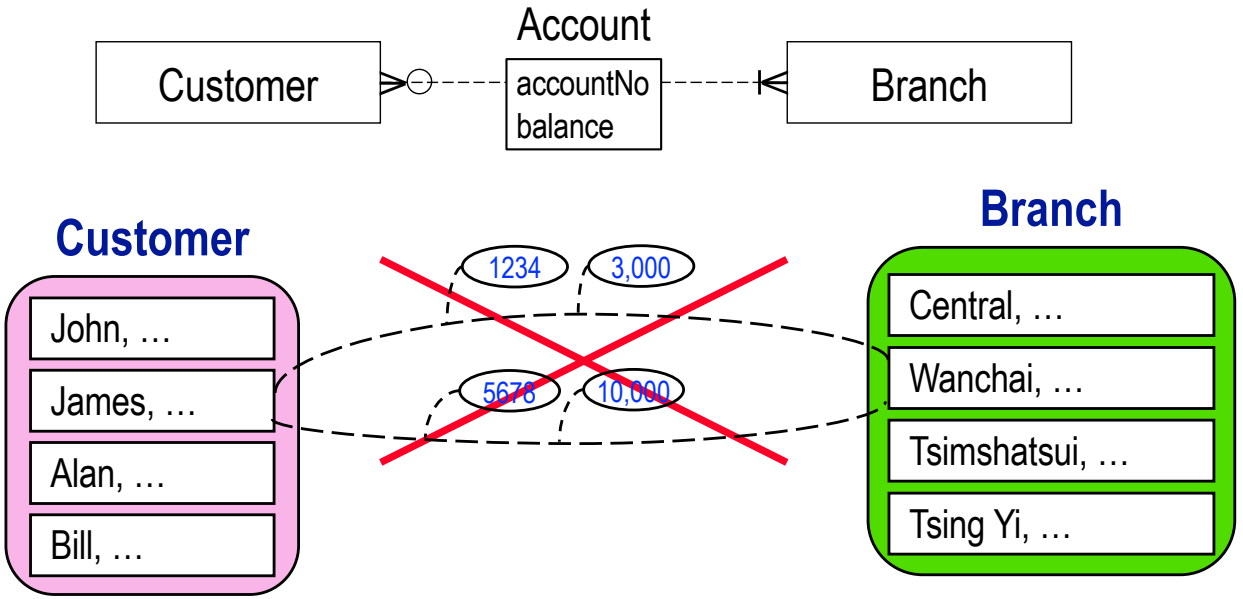

## Account as relationship




**What if you want to have several accounts for a customer at the same branch?**

# DESIGN CHOICE: ENTITY VERSUS RELATIONSHIP (CONT'D)

**We want to represent the fact that James has two accounts at the same branch.**

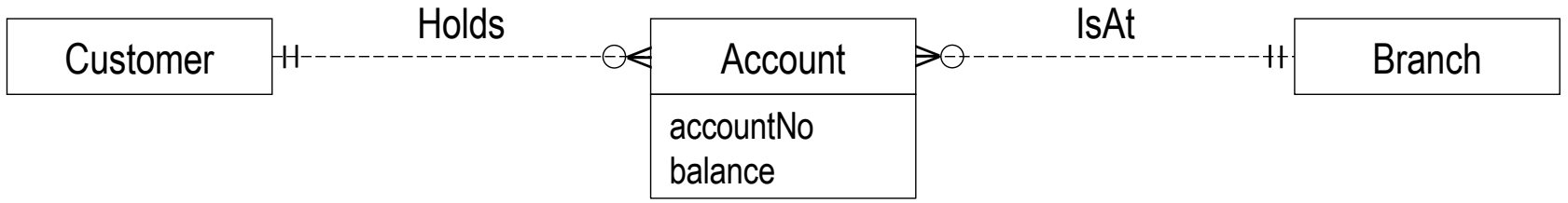


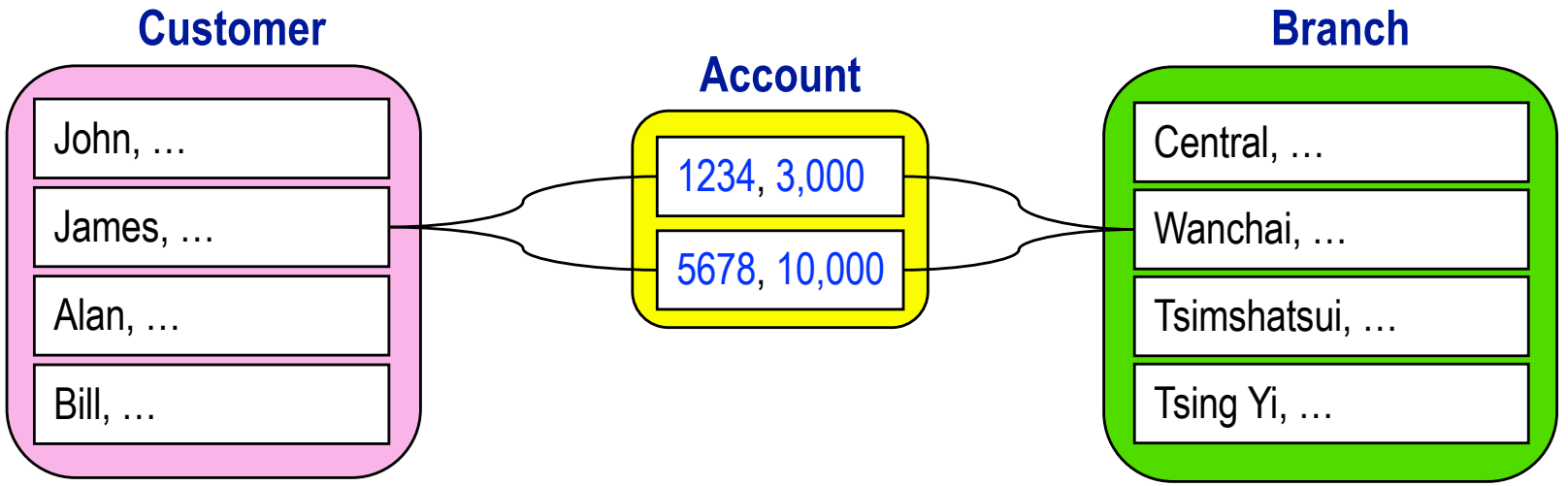

An given entity instance can be related to another given entity instance *at most once* by a given relationship.

A given Customer instance can be related to a given Branch instance *at most once* by the Account relationship.

# DESIGN CHOICE: ENTITY VERSUS RELATIONSHIP (CONT'D)

**We need to use an entity for Account!**

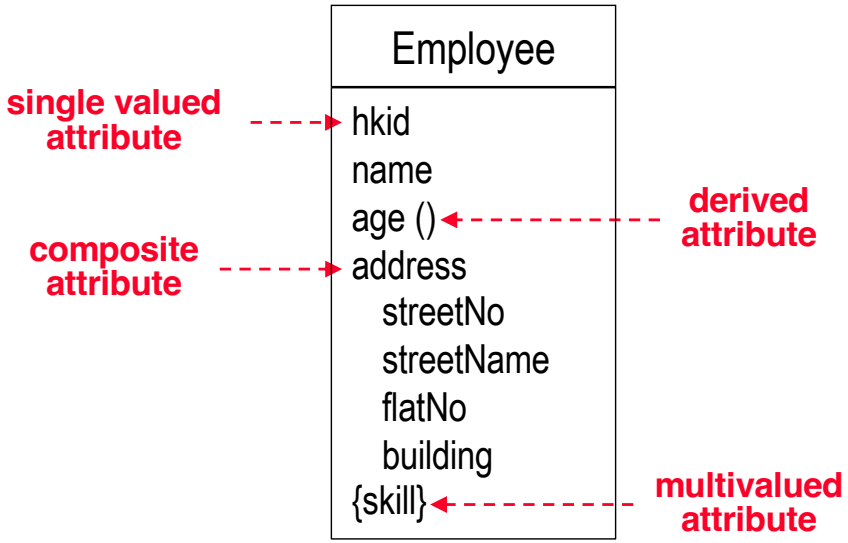



**There can be only one relationship instance of a given relationship type between the same two entity instances.**

# E-R DATA MODEL CROW FOOT NOTATION

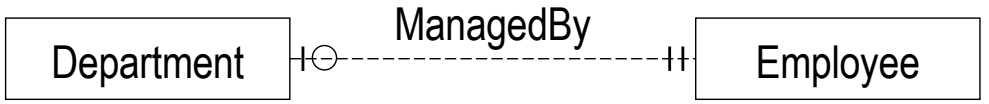
**entity and attribute**



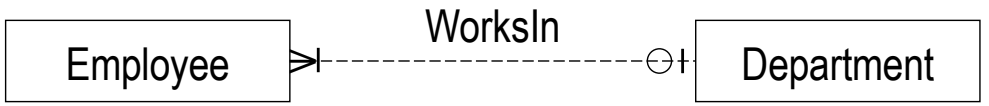

**relationship**

○ - partial (min-card = 0)  I - total (min-card = 1)

**(a) one-to-one (1:1)**



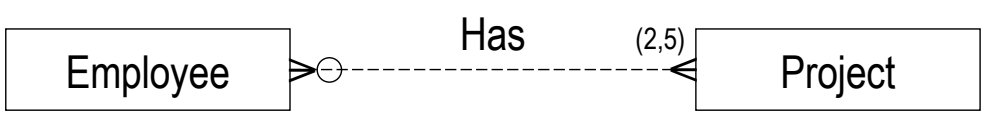

**(b) one-to-many (1:N)**



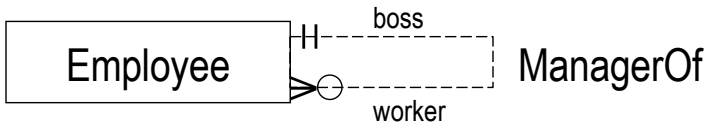
# E-R DATA MODEL CROW FOOT NOTATION (CONT'D)

**c) many-to-many (N:M)**

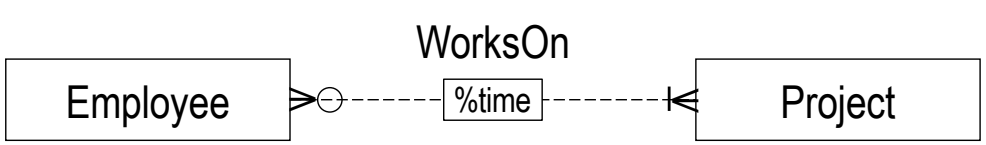

**d) explicit cardinality**




**e) recursive (with role names)**

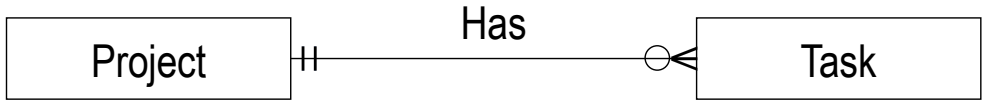



**f) relationship attribute**

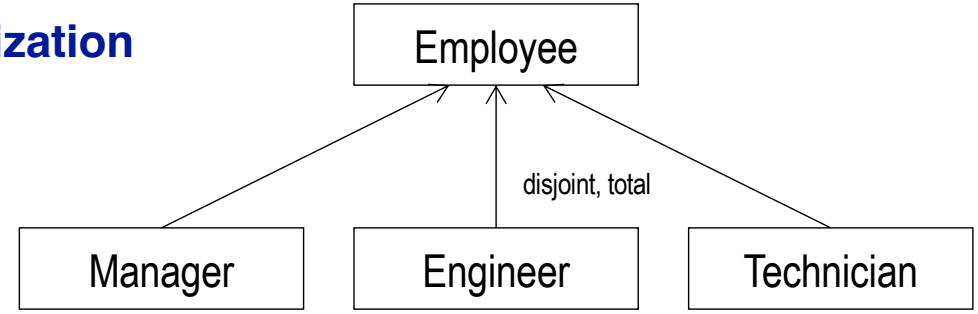


# E-R DATA MODEL CROW FOOT NOTATION (CONT'D)

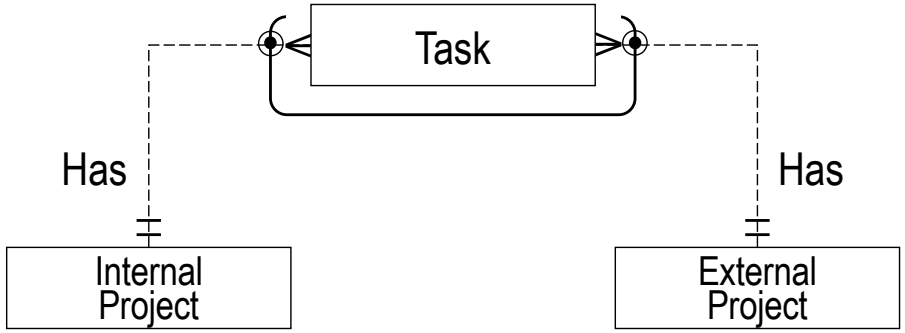
**g) identifying relationship**




**h) generalization/specialization**




**i) exclusion constraint**



# DATABASE DESIGN: SUMMARY

- **Database design** is the process of capturing data requirements from an application domain and designing a database that represents these requirements.

- On the one hand, the E-R data model is often used to represent and document data requirements due to its expressiveness and graphical notation.

- On the other hand, the relational data model is widely used to implement data requirements in a database managed by a relational DBMS.